

GM University

Course Code: UE26CS1103

Course Title: C Programming with Practice

Project-Based Learning (PBL) Report

Student Academic Record and Performance Management System using C

Assigned PBL title (same project for all teams)

Team Members

S.No	Name	Roll Number	Role in the project
1	[Member 1 name]	[roll no.]	[e.g. input handling and program flow]
2	[Member 2 name]	[roll no.]	[e.g. records, display and search]
3	[Member 3 name]	[roll no.]	[e.g. calculations and analysis]
4	[Member 4 name]	[roll no.]	[e.g. testing and documentation]

Faculty / Course instructor	Semester / Section	Date of submission
[faculty name]	[semester / section]	[dd/mm/yyyy] (within Week 14)

Maximum marks: 40 | Duration: 14 weeks | Group size: at most 4 students

Report: one per group, maximum 12 A4 pages

1. Abstract

Academic departments constantly record marks, compute results and look for weak areas; doing this by hand is slow and error-prone. This project develops a menu-driven console program in C that stores student academic records, validates every input, calculates total, average, percentage and a pass/fail decision, searches by identifier or name, and reports highest/lowest performance, mark-band frequencies and pass/fail counts. Identity data is held in an array of structures and the marks in a separate two-dimensional student-by-subject table; 58 user-defined functions in five source modules implement the operations. Keyboard input is read with `fgets` and converted with `strtol`, so invalid, over-long and out-of-range input is rejected without corrupting stored data. Six optional menu options were added: update marks, ranking, partial-name search, save, load and CSV export. The system was verified with 139 unit checks and 45 scripted integration cases covering normal, boundary and invalid input; all pass, including under AddressSanitizer and UndefinedBehaviorSanitizer. Records live in memory unless saved, and capacity is fixed at 100 students.

2. Introduction

Every institution keeps academic records: who a student is, what they scored in each subject, whether they passed, and how the class performed. Even a small system for this task needs decisions (does the student pass?), repetition (every student, every subject), tables (marks by subject), text handling (names and identifiers) and careful input checking. That makes it a good vehicle for applying the first six units of the course, **C Programming with Practice**, in one coherent program.

The purpose of this project is to design, implement, test and demonstrate such a system in standard C (C11), as a terminal application. Records are kept in memory, with optional text-file persistence added as an enhancement. The program is about 1835 lines of commented C across five modules and compiles without warnings under `-Wall -Wextra -Wpedantic`. This report follows the prescribed structure: problem statement, objectives and requirements, algorithms and flowcharts, system design, implementation, testing and results, conclusion, references and the individual contribution table.

3. Problem Statement

Design and develop a menu-driven C programming system to maintain student academic records and perform useful computations and searches. The system should integrate iterative programming, arrays, strings, two-dimensional arrays, structures and user-defined functions.

The system therefore has to hold for each student an identifier, a name and one mark per subject; accept and **validate** that data; display it in readable form; calculate the total, average and percentage; decide pass or fail; **search** by identifier or name; and **analyse** the class (highest and lowest results, frequency of mark bands, pass/fail counts). The assignment does not fix the numbers and rules involved, so the following design choices were made and are used consistently in the code, tests and this report:

Design choice	Value used	Design choice	Value used
Maximum records	100 students	Pass rule	every subject mark ≥ 40
Subjects	5 (Maths, Physics, Chemistry, English, Computing)	Frequency bands	0–39, 40–59, 60–79, 80–100 per subject
Marks	whole numbers 0–100	ID	unique, 1–19 characters, no spaces, case-sensitive
Name	1–79 characters, spaces allowed, duplicates allowed	Search	exact ID; exact (case-sensitive) name, all matches

4. Objectives and Functional Requirements

Objectives. (1) Apply problem-solving and programming fundamentals (Units 1–2). (2) Use decision-making and user-defined functions (Unit 3). (3) Implement iterative algorithms and number processing (Unit 4). (4) Use arrays and strings for collections and text (Unit 5). (5) Use two-dimensional arrays and structures for tables and records (Unit 6).

Functional requirements and how each is met (menu options 1–8 are the required features; 9–14 are enhancements):

Requirement	Implementation	Test evidence
Data entry and validation	Option 1 <code>add_student</code> ; <code>read_int_in_range</code> , <code>read_nonempty_text</code> , <code>duplicate-ID</code> check, <code>capacity</code> check	TC01, TC11–TC12, TC20–TC28
Formatted display	Option 2 <code>aligned</code> table; <code>display_student</code> detailed view	TC02
Total, average, percentage	<code>calculate_total</code> , <code>calculate_average</code> , <code>calculate_percentage</code> (floating-point)	TC01, TC02, unit tests
Pass/fail decision	<code>is_pass</code> : every subject ≥ 40	TC13, TC14
Search by ID or name	Option 3 (<code>find_student_by_id</code>), option 4 (<code>search_by_name</code>)	TC03–TC05
Maximum/minimum and frequency	Option 6 <code>show_extremes</code> (ties listed), option 7 <code>show_frequency</code> , option 8 <code>pass/fail</code>	TC07–TC09, TC16–TC19
Subject-wise marks	Two-dimensional <code>marks[100][5]</code> ; option 5 table with class averages	TC06
Structures	<code>Student</code> (<code>id</code> , <code>name</code>) and <code>Records</code> (<code>students</code> , <code>marks</code> , <code>count</code>)	Section 6
Reusable functions	58 functions, prototypes in <code>student.h</code> ; <code>main</code> only dispatches	Section 7
Testing	Normal, boundary and invalid cases	Section 8, <code>tests/test-cases.md</code>
Enhancements (options 9–14)	Update marks, ranking, partial-name search, save, load, CSV export	TC35–TC45

5. Algorithm / Flowchart

5.1 Overall program flow

The program initialises an empty record table and then repeats: show the menu, read a valid choice, run the chosen operation through its function, and return to the menu, until Exit is chosen or the input ends (Figure 1).

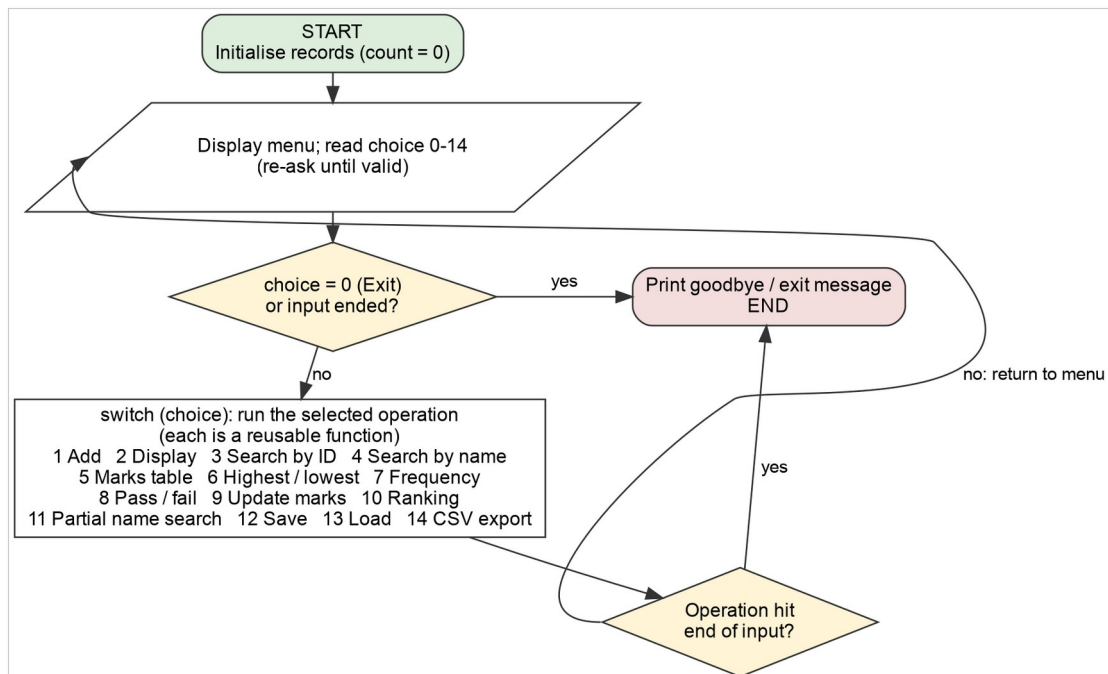


Figure 1: Main program flow (menu loop)

5.2 Entry validation and result calculation

Algorithm: add one student

1. If the table already holds 100 students, report it and return.
2. Read the ID; re-ask if blank, over 19 characters or containing spaces.
3. If the ID is already stored (linear search with `strcmp`), reject it and return.
4. Read the name; re-ask if blank or over 79 characters.
5. For each of the 5 subjects read a mark with `strtoul`; re-ask for text, trailing characters, overflow, below 0 or above 100.
6. If the input ends at any prompt, discard the temporary data and return.
7. Only now copy the temporary identity and marks into row `count` and increase `count` by one.
8. Display the saved record with its calculated result.

The new data stays in temporary variables until step 7, so rejecting a duplicate or cancelling can never leave a half-written record or a wrong count.

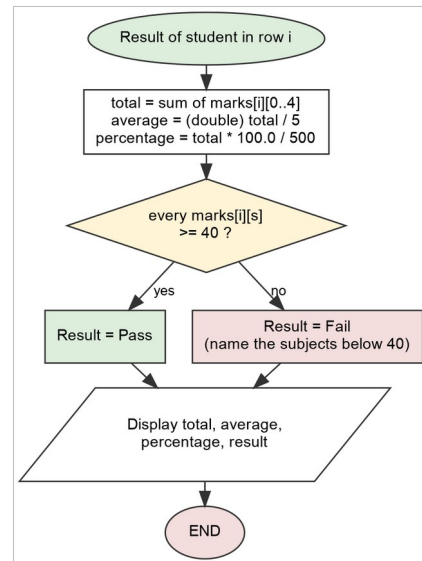


Figure 2: Calculation of total, average, percentage and result

5.3 Key operation logic

- **Search.** By ID: compare the wanted ID with `students[0].id`, `students[1].id`, ... using `strcmp` until a match or the end (linear search). By name: the same, but the whole table is always traversed because names are not unique.
- **Highest/lowest.** Stop if there are no records; start both from the first real record, then compare the remaining rows; finally list every student that holds the extreme value (ties). The same is done for each subject column.
- **Frequency.** Zero a `counts[subject][band]` table; for every mark add one to exactly one band (80+ → 3, 60+ → 2, 40+ → 1, else 0); every subject row must add up to the number of students.
- **Pass/fail.** Count students for whom `is_pass` is true or false; `pass percentage = passes × 100 / students`.

6. System Design / Methodology

Approach. The system was designed top-down and built incrementally in the order: constants and data model, validated input and menu, entry and display, calculations, search, analysis, integration, then testing. Hand-calculated expected results (for example S001: 80+70+60+90+50 = 350, $350 \div 5 = 70.00$, $350 \times 100 \div 500 = 70.00\%$) were written down before the code and every function was checked against them.

Data model. One `Records` structure holds an array of `Student` structures (`id`, `name`), a separate two-dimensional `marks` table and the number of occupied rows. Row `i` of both arrays describes the same person (Figure 3). Marks are stored once; total, average, percentage and result are recalculated whenever they are needed, so they can never be stale after an update.

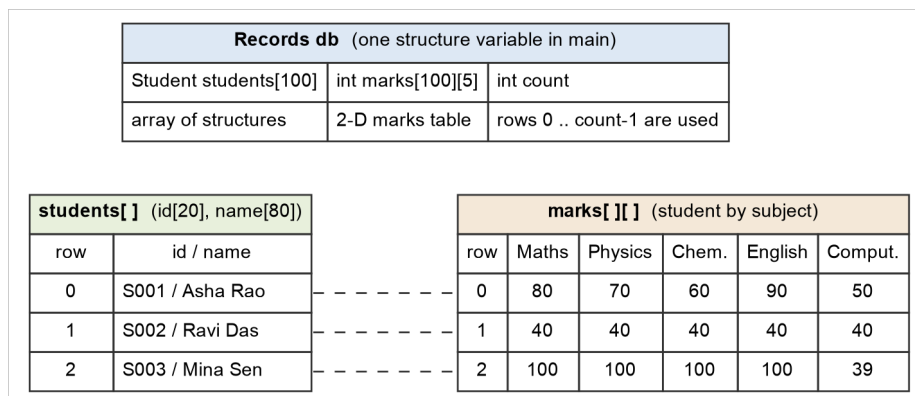


Figure 3: Data model — identity structures and the two-dimensional marks table share a row index

Module	Responsibility	Main functions
main.c	Menu, dispatch with switch, clean exit on end of input	main, print_menu
student.h	Constants, Student and Records types, all prototypes	—
student.c	Entry, display, search, marks table, update marks	add_student, display_student, find_student_by_id, search_by_name, display_marks_table
analysis.c	Calculations, decisions, extremes, frequency, summary, ranking	calculate_total, is_pass, mark_band, show_extremes, show_frequency
input.c	Safe bounded input	read_int_in_range, read_nonempty_text, parse_int_in_range
storage.c	Optional save/load and CSV export	save_records, load_records, export_csv

Academic rules and formulas. With $S = 5$ subjects and a maximum of $M = 100$ per subject: $\text{total} = \text{sum of the marks}$; $\text{average} = \text{total} / S$; $\text{percentage} = \text{total} \times 100 / (S \times M)$, both in floating point; $\text{pass} = \text{every subject mark} \geq 40$. A student with 100, 100, 100, 100, 39 has 87.80 % but **fails**, and the program names the subject responsible.

Design decisions.

- **Input policy.** Every keyboard read goes through `input.c`: read a line with `fgets`, discard the rest of an over-long line, trim spaces, convert with `strtoul` (clearing `errno` first) and reject empty text, letters, 42abc, 4.5, overflow and out-of-range values; end of input cancels the current operation cleanly. `scanf` is never used.
- **Atomic changes.** Adding builds the record in temporaries and commits it last; updating edits a temporary copy of the marks; loading parses into a scratch table and replaces memory only if the whole file is valid.
- **Const-correct data.** Marks and students are wrapped in `Records` so read-only functions take `const Records *`. Passing a bare `int[][5]` to a `const int (*)[5]` parameter is rejected by GCC under `-std=c11 -Wpedantic` (“pointers to arrays with different qualifiers”), which the wrapper avoids.
- **Ranking without moving data.** Ranking sorts an array of row indices, so the student and marks rows can never get out of step.

7. Implementation

The program consists of 58 functions, 69 `for` loops, 7 `while` loops and 2 `switch` statements. The excerpts below are copied from the source files.

Data types (`student.h`): an array of structures and a two-dimensional array inside one structure.

```
typedef struct {
    Student students[MAX_STUDENTS];
    int marks[MAX_STUDENTS][SUBJECT_COUNT];
    int count;
} Records;
```

Pass rule (`analysis.c`): iteration with an early exit and a decision; the function is reused by every report.

```
/* Academic rule: pass only when EVERY subject mark is at least PASS_MARK. */
bool is_pass(const int row[SUBJECT_COUNT])
{
    int s;

    for (s = 0; s < SUBJECT_COUNT; s++) {
        if (row[s] < PASS_MARK) {
            return false;
        }
    }
    return true;
}
```

Safe numeric input (`input.c`): `errno` is cleared, `strtol` is used, and a partial match such as `42abc` is rejected.

```
/*
 * Convert text to an int in [min, max] without accepting partial matches.
 * errno is cleared first because strtol never clears it (see C library docs).
 */
ParseResult parse_int_in_range(const char *text, int min, int max, int *value)
{
    char *end;
    long parsed;

    if (text == NULL || text[0] == '\0' || isspace((unsigned char)text[0])) {
        return PARSE_NOT_NUMBER;
    }

    errno = 0;
    parsed = strtol(text, &end, 10);

    if (end == text || *end != '\0') {
        return PARSE_NOT_NUMBER;          /* nothing converted, or "42abc" */
    }
    if (errno == ERANGE || parsed < min || parsed > max) {
        return PARSE_OUT_OF_RANGE;        /* overflow or outside the range */
    }
    *value = (int)parsed;
    return PARSE_OK;
}
```

Committing a record last (`student.c`, end of `add_student`): reached only after every field has been validated.

```
/* Everything is valid: commit the whole record, then count it once. */
db->students[db->count] = fresh;
for (s = 0; s < SUBJECT_COUNT; s++) {
    db->marks[db->count][s] = fresh_marks[s];
}
db->count++;
```

Frequency analysis (`analysis.c`): a two-dimensional counter table is zeroed and then filled by nested loops.

```
/* counts[s][b] = number of students whose mark in subject s is in band b. */
void compute_band_counts(const Records *db, int counts[SUBJECT_COUNT][BAND_COUNT])
{
    int s;
    int b;
    int i;

    for (s = 0; s < SUBJECT_COUNT; s++) {
        for (b = 0; b < BAND_COUNT; b++) {
            counts[s][b] = 0;          /* initialise every counter */
        }
    }
    for (i = 0; i < db->count; i++) {
        for (s = 0; s < SUBJECT_COUNT; s++) {
            counts[s][mark_band(db->marks[i][s])]++;
        }
    }
}
```

Course-unit coverage in the code:

Unit	Where it appears
1–2 Fundamentals, problem solving	Data types, expressions, formatted I/O, algorithms in Section 5, compilation with warnings enabled
3 Decisions and functions	Menu switch, validation, <code>is_pass</code> , <code>if/else</code> chains; every operation is a function with a prototype
4 Iteration, number processing	Totals, minimum/maximum, counting, frequency bands, insertion sort in <code>show_ranking</code>
5 Arrays and strings	<code>id</code> and <code>name</code> arrays, <code>strcmp/strlen/snprintf</code> , band counters, case-insensitive substring search

Unit	Where it appears
6 2-D arrays and structures	marks[100][5], Student, Records, column sums for class averages

8. Testing, Results & Discussion

Method. Expected values were hand-calculated first (five students S001–S005; Section 6). Testing has three layers: (1) 139 unit checks (`tests/unit_tests.c`) on calculations, band edges, parsing, validators, search, entry, update and file handling; (2) 45 scripted integration cases that type a fixed keystroke script into the real program and compare the transcript with the expected text — each also fails on a non-zero exit code or any output on `stderr`; (3) robustness runs. Table 1 lists representative cases; the “Actual output” column quotes lines from the transcripts of the run (spacing condensed). All 45 cases and their full transcripts are in `tests/test-cases.md` and `tests/transcripts/`.

ID	Category	Input / action	Expected	Actual output (from the run)	Result
TC01	Normal	Add S001: 80 70 60 90 50	Total 350/500, average 70.00, 70.00 %, Pass	Total : 350 / 500 Average : 70.00 Percentage : 70.00% Result : Pass	Pass
TC03	Normal	Search ID S004	Omar Ali's record, total 0, Fail	Name : Omar Ali Total : 0 / 500 Result : Fail	Pass
TC04	Normal	Search name “Asha Rao”	Two records (S001, S005)	Found 2 record(s) named "Asha Rao".	Pass
TC05	Normal	Search missing ID S999 and name “Nobody Here”	Clear “not found” messages	No student found with ID "S999". No student found with the name "Nobody Here".	Pass
TC07	Normal	Highest / lowest performance	S005 highest (500), S004 lowest (0)	Highest total: 500 / 500 (100.00%) Lowest total: 0 / 500 (0.00%)	Pass
TC08	Normal	Frequency analysis	Maths bands 1, 1, 0, 3 (total 5)	Maths 1 1 0 3 5	Pass
TC09	Normal	Pass/fail summary	3 pass, 2 fail, 60.00 %	Passed : 3 Failed : 2 Pass percentage : 60.00%	Pass
TC13	Boundary	All marks exactly 40	Pass (40 ≥ 40)	Average : 40.00 Result : Pass	Pass
TC14	Boundary	Marks 100 100 100 100 39	Fail although 87.80 %	Percentage : 87.80% Result : Fail	Pass
TC15	Boundary	Options 2–11 and 14 with no records	Safe message each time, no crash	No student records available yet. Use option 1 to add students. (shown 11 times)	Pass
TC17	Boundary	Two students tied for highest	Both listed, tie noted	Highest total: 450 / 500 (90.00%) - 2 students tied	Pass
TC18	Boundary	100th record, then a 101st attempt	100th accepted; 101st rejected, count stays 100	Student added successfully (100 of 100 records used). Capacity reached: the table already holds 100 students. Nothing was added. 100 student record(s) shown. Pass rule: every subject >= 40.	Pass
TC19	Boundary	Maths 39, 40, 59, 60, 79, 80	Bands 1, 2, 2, 1 (total 6)	Maths 1 2 2 1 6	Pass
TC20	Invalid	Mark -1, then 101	Rejected and asked again	-1 is out of range. Enter a whole number from 0 to 100. 101 is out of range. Enter a whole number from 0 to 100.	Pass

ID	Category	Input / action	Expected	Actual output (from the run)	Result
TC21	Invalid	Mark abc, 42abc, 4.5, blank	All rejected; 42abc is not read as 42	"abc" is not a valid whole number. Enter a whole number from 0 to 100. "42abc" is not a valid whole number. Enter a whole number from 0 to 100. "4.5" is not a valid whole number. Enter a whole number from 0 to 100. input cannot be blank. Enter a whole number from 0 to 100.	Pass
TC22	Invalid	Mark 99999999999999999999	Rejected as out of range	99999999999999999999 is out of range. Enter a whole number from 0 to 100.	Pass
TC23	Invalid	Duplicate ID S001	Rejected; original record and count kept	ID "S001" already exists. The record was not added. Records stored: 1 / 100	Pass
TC25	Invalid	ID of 25 and 300 characters, name of 90	Length errors; later input not corrupted	input is too long (maximum 19 characters, you typed 25). input is too long (maximum 79 characters, you typed 90). Student added successfully (1 of 100 records used).	Pass
TC26	Invalid	Menu choices 99, -1, abc, 3.5, blank	Message, prompt returns, normal exit	99 is out of range. Enter a whole number from 0 to 14. Thank you for using the system. Records held in memory are lost on exit	Pass
TC27	Invalid	Input ends in the middle of an entry	No partial record; clean exit	Input ended. The record was not added. Exiting the program.	Pass

Table 1: Representative test cases (normal, boundary, invalid)

Sample output. Menu option 2 for the hand-calculated dataset (S003 fails with 87.80 %, S002 passes on the boundary), and option 8:

```

=== All Student Records ===
ID      Name      Math  Phys  Chem  Engl  Comp  Total  Average  Percent  Result
-----
S001    Asha Rao      80    70    60    90    50    350    70.00    70.00%    Pass
S002    Ravi Das       40    40    40    40    40    200    40.00    40.00%    Pass
S003    Mina Sen       100   100   100   100   39    439    87.80    87.80%    Fail
S004    Omar Ali        0     0     0     0     0     0     0.00    0.00%    Fail
S005    Asha Rao      100   100   100   100   100    500   100.00   100.00%    Pass

=== Pass / Fail Summary ===
Pass rule      : every subject mark must be at least 40
Total students : 5
Passed        : 3
Failed        : 2
Pass percentage : 60.00%

Failed students (with the subjects below 40):
  S003  Mina Sen   87.80%  below pass mark in: Computing (39)
  S004  Omar Ali   0.00%  below pass mark in: Maths (0), Physics (0), Chemistry (0), English (0),
  Computing (0)

```

Figure 4: Captured console output (tests TC02 and TC09)

Verification summary.

Check	Result
Unit tests	139 checks, 0 failed
Integration tests	45 cases, 45 passed, 0 failed (normal, boundary, invalid and enhancement cases)
Compiler warnings	None with <code>-std=c11 -Wall -Wextra -Wpedantic</code> ; also clean with <code>-Wshadow -Wconversion -Wformat=2</code>

Check	Result
Memory/undefined behaviour	All unit and integration tests pass under AddressSanitizer and UndefinedBehaviorSanitizer
Randomised input	500 random sessions (400 random menu/text sessions and 100 raw random byte streams) against the sanitizer build: no crash, no sanitizer report
Do the tests detect bugs?	Two mutation-testing rounds of 13 deliberately injected bugs each (for example <code><=</code> instead of <code><</code> in the pass rule, integer division for the average, removed duplicate check, removed capacity check, <code>42abc</code> accepted): the unit suite caught 12 of 13 (the remaining mutant is equivalent on 64-bit systems) and the integration suite caught 13 of 13 after test TC25 was strengthened

Discussion. The program reproduced every hand-calculated value, including the key distinction that a student can have a high percentage and still fail (S003: 87.80 %, Fail) and that a mark exactly on the boundary passes (S002). Boundary behaviour — empty data, a single record, ties, the 100th and 101st record and the band edges 39/40, 59/60, 79/80 — matched the expected results, and no invalid input caused a crash, an endless loop or a changed record count. Checking the tests themselves found two test-side issues, both fixed: a column-spacing mistake in the expected text of one enhancement test (the program was right), and a gap that let a missing “discard the rest of an over-long line” step go unnoticed (TC25 now detects it); reviewing the output also led to removing trailing spaces from table rows. No functional defect was found in the final program by any of these checks. Limitations observed: records are lost on exit unless saved, capacity (100) and subject count (5) are fixed at compile time, exact searches are case-sensitive, and table columns are aligned by bytes, so non-ASCII names may look misaligned.

9. Conclusion & Future Scope

The project delivers a complete menu-driven C system that meets every functional requirement: validated entry, formatted display, total/average/percentage, a pass/fail rule, search by identifier and name, extremes and frequency analysis, subject-wise marks in a two-dimensional table, structures for records, reusable functions, and documented normal/boundary/invalid testing. It demonstrates all six course units in one program, and the safety-first input handling and “commit last” updates keep the data consistent even when the user makes mistakes or the input ends unexpectedly.

Future scope: dynamic memory (`malloc`) for an unlimited number of students; configurable subjects and per-subject maximum marks; grade bands and CGPA; sorting by any column and richer search (by range of marks); a binary or database back end with backups; separate teacher/student views with authentication; and a graphical or web front end reusing the same calculation modules.

10. References

- [1] GM University, C Programming with Practice (UE26CS1103): Project-Based Learning (PBL) guidelines, deliverables, report format, rubrics and timeline, course document, pp. 15–19.
- [2] cppreference.com, “`strtol`, `strtoll`” (C standard library reference), <https://en.cppreference.com/w/c/string/byte/strtol> — accessed 8 October 2026.
- [3] cppreference.com, “`fgets`” (C standard library reference), <https://en.cppreference.com/w/c/io/fgets> — accessed 8 October 2026.
- [4] Y. Shafranovich, “Common Format and MIME Type for CSV Files”, RFC 4180, IETF, October 2005, <https://www.rfc-editor.org/rfc/rfc4180> — accessed 8 October 2026.

11. Individual Contribution Table

Individual contribution declaration. Use only real members; percentages must reflect the work actually done and total 100 %. Every member signs.

Name	Role	Contribution %	Signature
[Member 1 name]	[work actually performed]	[%]	
[Member 2 name]	[work actually performed]	[%]	
[Member 3 name]	[work actually performed]	[%]	
[Member 4 name]	[work actually performed]	[%]	

Name	Role	Contribution %	Signature
Total		100 %	

Suggested split of the work areas in the source: input handling and program flow (`input.c`, `main.c`); records and searching (`student.c`); calculations and analysis (`analysis.c`); testing, documentation and enhancements (`tests/`, `storage.c`, `report`). Replace this note with each member's real contribution before submission.